# R1.05 - Introduction aux Bases de Données et SQL

## Partie manipulation de données

Yon Dourisboure

IUT de Bayonne, département informatique, 2026-2027



## Planning et objectif

- Planning:
  - 6 semaines
  - 4 séances de cours
  - 6 séances de TD
  - 6 séances de TP
- Objectifs:
  - Savoir manipuler une Base de Données de Relationnelle à l'aide du langage SQL :
    1. Création / modification du schéma relationnel (LDD)
    2. Ajout / modification / recherche de données (LMD)



## Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes



## Base de Données

Définition :

Ensemble organisé d'informations avec un objectif commun.

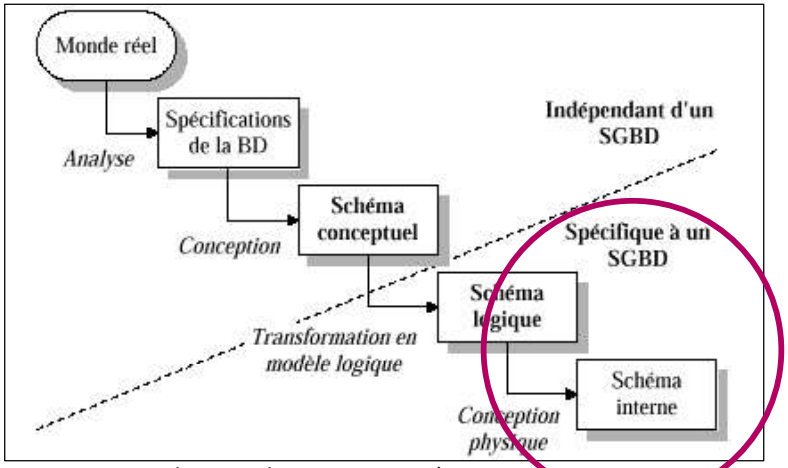
Plus précisément, on appelle base de données un ensemble structuré et organisé de façon à être facilement :

- Accessible (recherche d'informations),
- Géré
- Mis à jour

Concrètement :

Une base de données permet de mémoriser les objets et les liens entre les objets du monde réel à l'intérieur d'un système informatique.

# Processus de conception d'une BD



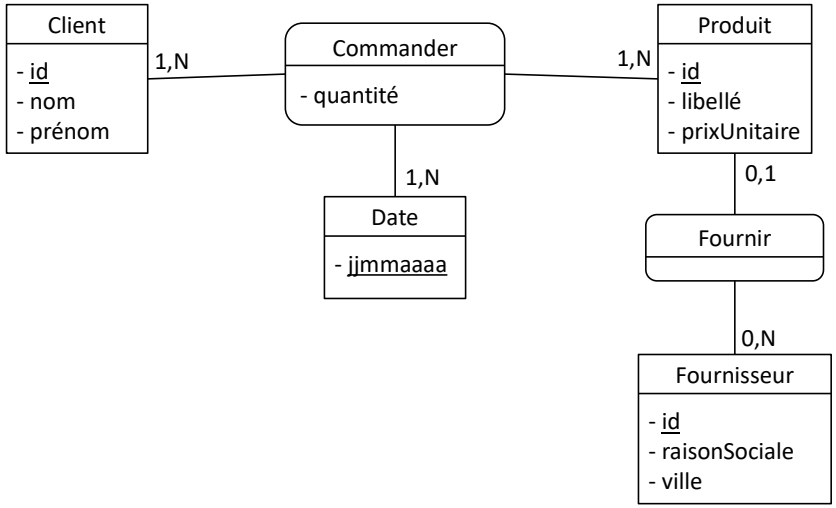

Extrait du cours de C.Marquesuzaà

Nous sommes ici



# Schéma conceptuel

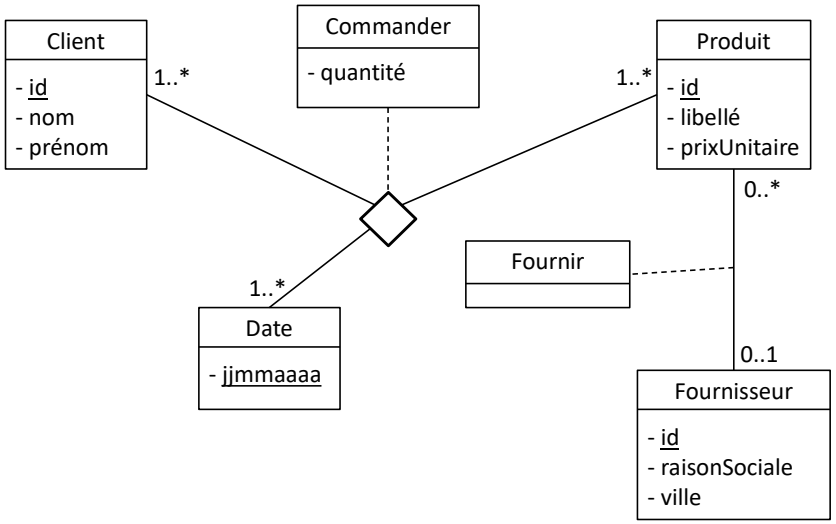
Format Entité / Association






# Schéma conceptuel

Format UML





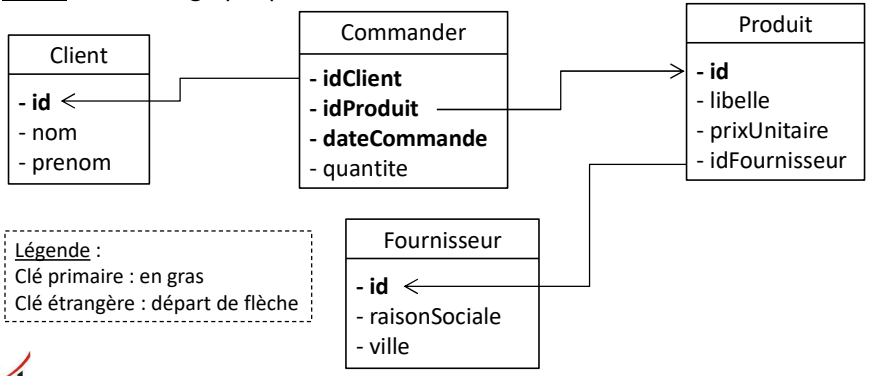

# Schéma relationnel / logique (1/2)

Liste des relations au format texte :

- Client(id, nom, prenom)
- Fournisseur(id, raisonSociale, siret)
- Produit(id, libelle, prixUnitaire, #idFournisseur)
- Commander(#idClient, #idProduit, dateCommande, quantite)

Légende :
Clé primaire : soulignée
Clé étrangère : précédée de #

**Et/ou** au format graphique :




Légende :
Clé primaire : en gras
Clé étrangère : départ de flèche

# Schéma relationnel / logique (2/2)

**Et** accompagné du dictionnaire des données :

Exemple avec la table Produit :

| Libellé | Signification | Domaine | | | Contraintes | Taille | Exemple |
|---|---|---|---|---|---|---|---|
| | | Conceptuel | Logique | Physique | | | |
| id | Code identifiant de chaque produit | Numérique | Entier | ? | Clé primaire | 5 | 12345 |
| libelle | Libellé de chaque produit | Alphabétique | Chaine de caractères | ? | - Obligatoirement renseigné | 25 | 'Stylo bleu' |
| prixUnitaire | Prix de vente de chaque produit en euros | Numérique | Réel à 2 décimal | ? | > 0 | ? | 2,50 |
| idFournisseur | Code du fournisseur qui approvisionne le produit | Numérique | Entier | ? | - Clé étrangère faisant référence au code unique du fournisseur | 5 | 12345 |



# Schéma physique

Une fois que les tables sont créées, il faut les **peupler** (les remplir), en ajoutant des **enregistrements** (des **lignes**) :

Produit

| id | libellé | prixUnitaire | idFournisseur |
|---|---|---|---|
| 1 | 'Stylo bleu' | 2,50 | 25 |
| 12 | 'Gomme' | 1,03 | 25 |
| 5 | 'Effaceur' | 3,20 | 57 |
| 8 | 'Marqueur' | NULL | NULL |
| ... | ... | ... | ... |

Contrainte d'intégrité référentielle

Fournisseur

| id | raisonSociale | ville |
|---|---|---|
| 12 | 'Entreprise fournisseur 12' | 'Bayonne' |
| 57 | 'Entreprise fournisseur 57' | 'Anglet' |
| 25 | 'Entreprise fournisseur 25' | 'Anglet' |
| ... | ... | ... |



# Outils

Système de Gestion de Base de Données (SGBD) :

Logiciel système servant à stocker, à manipuler ou gérer, et à partager des données dans une base de données, en garantissant la qualité, la pérennité et la confidentialité des informations, tout en cachant la complexité des opérations. Exemple de ceux que vous utiliserez à l'IUT :

- **Oracle**
- MySQL
- MongoDB

} **SGBD Relationnels**

Structured Query Language (SQL) «Langage de requêtes structurées » :

Langage informatique normalisé servant à exploiter des bases de données relationnelles. On distingue plusieurs niveaux :

- Langage de Définition des Données (**LDD**)
- Langage de Manipulation des Données (**LMD**)
- Langage de Contrôle des Données (LCD)



# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes

# Vocabulaire

- Une relation devient une **Table**
- Dans une table on retrouve :
  - Des **champs (colonnes)** correspondant aux attributs de la relation
  - Des contrainte d'intégrités (héritées du niveau relationnel) :
    - > Contraint de **type**
    - > Contrainte de **domaine** (restriction vis-à-vis du type : intervalle, liste de valeurs, contraintes en valeurs par comparaison, etc.)
    - > Contrainte d'**existence**
    - > Contrainte d'**unicité**
    - > Contrainte d'**intégrité référentielle**
  - Des **enregistrements (lignes, tuples)**
    - > Remplir la table = **Peupler**
    - > les valeurs des champs de chaque enregistrement respectent les contraintes



# Création d'une table (LDD)

```
CREATE TABLE NomTable (
    nomChamp1 type [contrainte],
    nomChamp2 type [contrainte],
    …
    PRIMARY KEY (listeChamps),
    FOREIGN KEY (listeChamps)
        REFERENCES NomTable(listeChamps) [options],
    FOREIGN KEY (listeChamps)
        REFERENCES NomTable(listeChamps) [options],
    …
)
```

Avec :

- `type ∈ { NUMBER(taille), NUMBER(taille, nbDecimales), VARCHAR2(taille), DATE, BOOLEAN, etc}`
- `contrainte ∈ { UNIQUE, NULL, NOT NULL, CHECK(condition), DEFAULT valeur, etc.}`
- `options : vous verrez plus tard…`



# Création d'une table

Exercices :

1. Complétez le dictionnaire des données du schéma relationnel de la diapositive n°8
2. Donnez toutes les requêtes de création des tables
3. Quelle précaution faut-il prendre lors de la création des table ?



# Modification / Suppression d'une table (LDD)

- Modification de la structure d'une table :

```
ALTER TABLE NomTable (
    MODIFY nomChamp type [contrainte],
    ADD nomChamp type [contrainte],
    DROP COLUMN nomChamp,
    …
)
```

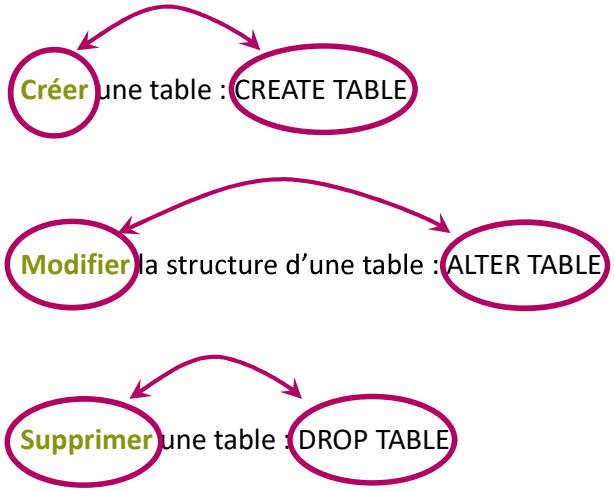
Remarque :

Il existe d'autres usages de ALTER, mais nous ne les utiliserons pas dans cette ressource

- Suppression d'une table :

```
DROP TABLE NomTable
```

# Synthèse du LDD






# Plan

1. Rappels
2. Du logique au physique (LDD)
3. **Requêtes de sélection « simples » de données(LMD)**
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes



# Requête de sélections simples

Afficher le contenu complet d'une table :

```
SELECT *
FROM NomTable
```

Opération de **projection** du contenu d'une table :

```
SELECT listeChamps
FROM NomTable
```

↳ Construit un tableau à partir d'un ou plusieurs champs

↳ On ne garde que les colonnes qui nous intéressent : **extraction verticale**

Exemple :

```
SELECT nom, prenom
FROM Client
```

↳ Affiche les noms et prénoms de tous les clients



# Requête de sélections simples

Opération de **restriction** du contenu d'une table : On choisit les **lignes** que l'on souhaite garder : **extraction horizontale** :

```
SELECT listeChamps
FROM NomTable
WHERE condition
```

↳ Seuls les **tuples** qui satisfont la condition seront conservés

Condition de sélection :

- Une expression simple : `nomChamp Opérateur valeur`
- Ou une combinaison d'expressions simples avec les opérateurs `AND, OR, NOT`

# Requête de sélections simples

Opérateurs pour les expressions simples de restriction :

- Opérateurs classiques : <, >, <=, >=, <>, !=
- IS NULL, IS NOT NULL : permet de déterminer si un champs contient ou pas une valeur
- LIKE, NOT LIKE : compare un champs avec une expression
- BETWEEN : vérifie si la valeur est dans un intervalle
- IN (listeValeurs), NOT IN(listeValeurs) : vérifie si la valeur est contenue dans un ensemble de valeurs

Exemple :

```
SELECT nom, prenom
FROM Client
WHERE (id BETWEEN 5 AND 20) AND (nom LIKE 'D%')
```

↳ Affiche les noms et prénoms des clients dont l'identifiant est compris entre 5 et 20 et dont le nom commence par la lettre 'D'



# Requête de sélections simples

Exercices.

Ecrire en langage SQL les requêtes permettant d'obtenir les résultats suivants (voir schéma relationnel diapo 8) :

1. La liste des produits dont le prix unitaire est supérieur à 10 €
2. La liste des identifiants des produits qui n'ont pas de prix
3. La liste des clients dont le nom est plus petit (ordre alphabétique) que le prénom.
4. La liste des fournisseurs (id, raison sociale) dont l'identifiant est plus grand que 10 et plus petit que 50



# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. **Manipulation des données (LMD)**
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes



# Insertion d'enregistrements (LMD)

```
INSERT INTO NomTable
VALUES (listeValeurs)
```

La liste de valeur doit contenir une valeur pour tous les champs de la table et dans le bon ordre

```
INSERT INTO NomTable(listeChamps)
VALUES (listeValeurs)
```

La liste de champs doit être un sous ensemble des champs de la table et la liste de valeurs doit correspondre

# Mise à jour d'un enregistrement (LMD)

```
UPDATE NomTable
SET nomChamp1 = valeur, nomChamps2 = valeur, …
[ WHERE condition ]
```

↳ Met à jour tous les enregistrements / tuples / lignes qui satisfont la condition en modifiant les valeur des champs tel que spécifié

Exemple :

```
UPDATE Produit
SET prixUnitaire = 0
WHERE prixUnitaire IS NULL
```

↳ Affecte la valeur zéro au prix unitaire de tous les produits dont le prix unitaire n'était pas renseigné.



# Effacer un enregistrement (LMD)

```
DELETE FROM NomTable
[ WHERE condition ]
```

↳ Efface tous les enregistrements / tuples / lignes qui satisfont la condition

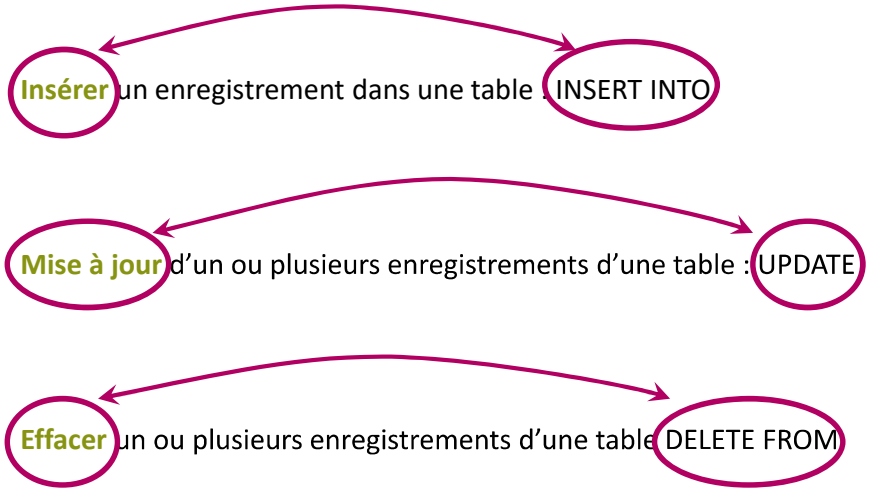
Exemple :

```
DELETE FROM Commander
WHERE quantite = 0
```

↳ Efface les enregistrements de la table Commander pour lesquels la quantité vaut zéro



# Synthèse de la manipulation de données






# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes

# Tri des résultats

Possibilité de trier les résultats selon 1 ou plusieurs champs :

```
SELECT …
FROM …
[ WHERE condition ]
[ ORDER BY nomChamp1 ASC/DESC, nomChamp2 ASC/DESC, … ]
```

Exemple :

```
SELECT *
FROM Client
ORDER BY nom ASC, id DESC
```

↳ Affiche tous les champs de tous les clients, les résultats étant triés par ordre alphabétique sur le nom et les homonymes triés par ordre décroissant de leurs identifiants.



# Opération entre champs

Possibilité de combiner les champs à l'aide d'opérateurs :

```
SELECT nomChamp1 Opérateur nomChamp2, …
FROM …
[ WHERE … ]
[ ORDER BY … ]
```

Avec les opérateurs mathématiques classiques +, -, /, * ainsi que le || pour la concaténation de chaines de caractères

Exemple :

```
SELECT id, nom || ' ' || prenom
FROM Client
```

↳ Affiche la liste des clients sous la forme d'une table à 2 colonnes : l'identifiant, puis le nom suivi d'un espace suivi du prénom



# Renommage des champs

Possibilité de renommer les champs le temps de la requête :

```
SELECT nomChamp AS nouveauNomChamp, …
FROM …
[ WHERE … ]
[ ORDER BY … ]
```

Notamment très utile lorsqu'on fait des opérations entre champs

Exemple :

```
SELECT id, nom || ' ' || prenom AS nomPrenom
FROM Client
```

↳ Même résultat que précédemment mais la seconde colonne aura comme étiquette « nomPrenom »



# Manipulation des dates

- SYSDATE permet de connaitre la date du jour
- L'unité de base d'un champ de type `date` est le jour.
- Il est possible d'ajouter ou de soustraire un nombre de jours
- La comparaison se fait normalement avec les opérateurs : `<, >, <=, >= et BETWEEN`
- `TO_DATE ('01/01/2026', 'DD/MM/AAAA')` renvoie la date correspondant au 1er janvier 2026
- `TO_CHAR (date, 'DD/MM/AAAA')` renvoie une chaine

Exemple :

```
SELECT idClient
FROM Commander
WHERE dateCommande >= SYSDATE - 1
```

↳ Renvoie tous les identifiants des clients ayant commandé quelques choses hier ou aujourd'hui

# Elimination des doublons

Possibilité de supprimer les doublons dans le résultat d'une requête :

```
SELECT DISTINCT …
FROM …
[ WHERE … ]
[ ORDER BY … ]
```

Exemple :

```
SELECT DISTINCT idClient
FROM Commander
WHERE dateCommande >= SYSDATE - 1
```

↳ Même résultat que précédemment mais sans répétitions



# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes



# La jointure

Problématique :

Afin d'éviter toute redondance, la conception de la base de donnée a « éclaté » les données : Un produit ne stocke que le numéro du fournisseur, les détails du fournisseur sont stockés dans la table Fournisseur.

Comment écrire une requête qui produit des lignes avec les caractéristiques d'un produit et de son fournisseur ?

On a besoin de « construire » un tableau regroupant les informations de plusieurs tables de manière efficace.

C'est l'opération de **jointure.**

Il existe plusieurs types de jointure, dans ce cours nous ne traiterons que de la **jointure interne.**



# La jointure

Principe :

La jointure de deux tables se fait en utilisant un champ (ou un ensemble de champs) dans chaque table dont les valeurs sémantiques sont semblables, typiquement une clé étrangère dans l'une et la clé primaire dans l'autre (mais pas nécessairement).

Le résultat est une nouvelle table dont :

- les champs correspondent à l'**union des champs** des deux tables d'origines. En cas d'ambiguïté on préfixe par le nom de la table d'origine : `NomTable.nomChamp`
- Les lignes correspondent à l'association de toutes les lignes ayant des valeurs identiques dans les champs choisis pour réaliser la jointure.

# La jointure

Exemple :

Jointure des tables Produit et Fournisseur grâce aux champs `idFournisseur` et `Fournisseur.id`

Produit

| id | libellé | prixUnitaire | idFournisseur |
|---|---|---|---|
| 1 | 'Stylo bleu' | 2,50 | 25 |
| 12 | 'Gomme' | 1,03 | 25 |
| 5 | 'Effaceur' | 3,20 | 57 |
| 8 | 'Marqueur' | NULL | NULL |
| … | … | … | … |

Fournisseur

| id | raisonSociale | ville |
|---|---|---|
| 12 | 'Entreprise fournisseur 12' | 'Bayonne' |
| 57 | 'Entreprise fournisseur 57' | 'Anglet' |
| 25 | 'Entreprise fournisseur 25' | 'Anglet' |
| … | … | … |

| id | libellé | prixUnitaire | idFournisseur | id | raisonSociale | ville |
|---|---|---|---|---|---|---|
| 1 | 'Stylo bleu' | 2,50 | 25 | 25 | 'Entreprise fournisseur 25' | 'Anglet' |
| 12 | 'Gomme' | 1,03 | 25 | 25 | 'Entreprise fournisseur 25' | 'Anglet' |
| 5 | 'Effaceur' | 3,20 | 57 | 57 | 'Entreprise fournisseur 57' | 'Anglet' |
| 8 | 'Marqueur' | NULL | NULL | … | … | … |
| … | … | … | … | … | … | … |



# La jointure

Syntaxe classique, en utilisant les alias pour renommer les tables :

```
SELECT …
FROM  NomTable1 T1 JOIN
      NomTable2 T2 ON T1.cleEtrangere = T2.clePrimaire
[ WHERE … ]
[ ORDER BY … ]
```

Remarques :

1. Le résultat de la jointure est une table, on peut donc en enchainer plusieurs à la suite.
2. Il existe d'autres types de jointures, tels que les jointures externes gauches et droites que vous verrez un peu plus tard…

Conseil :

La jointure est une opération commutative mais en pratique, commencez par la table contenant la clé étrangère, c'est plus facile pour « visualiser » le résultat : chaque ligne de la table de gauche produit au maximum une ligne dans la table résultat.



# La jointure

Exercices

Ecrire en langage SQL les requêtes permettant d'obtenir les résultats suivants (voir schéma relationnel diapo 8) :

1. La liste des produits (id, description) ainsi que la raison sociale de leur fournisseur.
2. La liste (identifiant, description) des produits commandés le 1er septembre 2026.
3. La liste (idClient, idProduit, dateCommande, cout) qui permet de savoir le coût (la quantité multipliée par le prix unitaire) de chaque ligne de la table `Commander`
4. La liste (nom, prénom) des clients ayant acheté des produits dont le prix unitaire dépasse les 5 euros.



# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes

# Les fonctions d'agrégation

Les fonctions d'agrégations permettent de produire un résultat à partir d'un ensemble de lignes.

Les plus courantes sont :

- COUNT(*) => renvoie le nombre de lignes
- COUNT(nomChamp) => renvoie le nombre de valeurs ≠ NULL
- SUM (nomChamp) => renvoie la somme
- AVG (nomChamp) => renvoie la moyenne
- MIN(nomChamp) => renvoie la plus petite valeur
- MAX(nomChamp) => renvoie la plus grande valeur

Exemple :

```
SELECT COUNT(*)
FROM Client
```

↳ Renvoie le nombre de clients



# La clause GROUP BY

Les fonctions d'agrégations s'utilisent généralement avec la clause GROUP BY afin de regrouper les lignes par catégorie

```
SELECT … , fonction1(…), fonction2(…), …
FROM …
[ WHERE … ]
GROUP BY nomChamp1, nomChamp2, …
```

↳ Toutes les lignes ayant les mêmes valeurs dans la champs

`nomChamp1, nomChamp2, ..`

sont regroupées en une seule ligne et les fonctions d'agrégations s'exécutent sur chaque sous-ensemble de lignes



# La clause GROUP BY

Exemple :

```
SELECT idFournisseur, COUNT(*) AS nbProduits
FROM Produit
GROUP BY idFournisseur
```

↳ Renvoie la liste (idFournisseur, nbProduits) correspondant au nombre de produits que fournit chaque fournisseur

Remarques :

1. Seuls les fournisseurs fournissant au moins 1 produit apparaissent
2. Tous les attributs présents dans la clause SELECT doivent être dans la clause GROUP BY : Il est interdit de demander à afficher le libellé du produit, en même temps cela n'aurait pas de sens…



# La clause HAVING

La clause HAVING permet de ne garder que les catégories vérifiant certains critères relatifs aux résultats de fonctions d'agrégation

```
SELECT …
FROM …
[ WHERE … ]
GROUP BY …
HAVING condition
```

- Exemple de conditions :
  - COUNT(*) > 2
  - MIN(nomChamp) = valeur
  - Ou même des combinaisons avec AND et OR
- Pas de conditions sur les champs simples, s'il y en a, elles doivent être dans la clause WHERE

# Synthèse sur les requêtes de sélection

Format général :

```
SELECT …
FROM …
[ WHERE … ]
[ GROUP BY … ]
[ HAVING … ]
[ ORDER BY … ]
```

Ordre d'évaluation :

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. ORDER BY
6. SELECT



# Synthèse sur les requêtes de sélection

Format général :

```
SELECT …
FROM …
[ WHERE … ]
[ GROUP BY … ]
[ HAVING … ]
[ ORDER BY … ]
```

Démarche de construction :

1. J'analyse la question posée pour identifier sur le schéma relationnel tous les champs qui vont être affichés et ceux dont on a besoin pour les restrictions exprimées dans la question.
2. J'en déduis toutes les tables et jointures dont j'ai besoin
3. J'écris ma requête dans le même ordre que son exécution :
   FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY -> SELECT



# Les fonctions d'agrégation

Exercices

Ecrire en langage SQL les requêtes permettant d'obtenir les résultats suivants (voir schéma relationnel diapo 8) :

1. Le nombre de fois que le client 1 a commandé le produit 12
2. La liste (idProduit, quantiteTotale) qui affiche, pour chaque produit que le client 1 a commandé, l'identifiant du produit et la quantité totale
3. La liste (ville, nbProduits) permettant de connaitre pour chaque ville le nombre de produits fournis par un fournisseur de la ville.
4. Le nom et le prénom des clients ayant acheté au moins 3 fois le produit d'identifiant 12



# Plan

1. Rappels
2. Du logique au physique (LDD)
3. Requêtes de sélection « simples » de données(LMD)
4. Manipulation des données (LMD)
5. Compléments sur les requêtes « simples » (LMD)
6. La jointure
7. Les requêtes avec agrégations
8. Les sous-requêtes

# Les sous-requêtes

En SQL il est tout a fait possible d'exécuter une requête à l'intérieur d'une autre, on parle alors d'une **sous-requête**

Dans ce cours la sous-requête sera systématiquement dans la clause WHERE ou HAVING de la requête principale…

Il existe 2 catégories de sous-requêtes :

1. Celles qui sont **indépendantes** : la sous-requête s'exécute indépendamment de la requête principale.
2. Celles qui sont **corrélées** : le résultat de la sous-requête dépend de la valeur d'un champs de la requête principale. La sous-requête doit être exécutée pour chaque tuple de la requête principale.



# Sous-requêtes indépendantes avec ANY

```
SELECT …
FROM …
WHERE nomChamp operateur ANY (
   SELECT nomChamp2
   FROM …
   …
)
…
```

Fonctionnement :

1. Le type retourné par la sous-requête doit correspondre à celui attendu par la requête principale
2. Le tuple de la requête principale sera gardé si le champ satisfait l'opérateur pour au moins une des valeurs renvoyées par la sous requête



# Sous-requêtes indépendantes avec ANY

Exemple : Lister les produits dont le fournisseur est à Bayonne

```
SELECT id, description
FROM Produit
WHERE prixUnitaire = ANY (
   SELECT id
   FROM Fournisseur
   WHERE ville = 'Bayonne'
)
```

Remarques :

1. Ce genre de requête remplace facilement les jointures lorsqu'on n'a pas besoin d'afficher des champs de la table « jointe »
2. Si la sous requête ne renvoie qu'un seul résultat on peut enlever le ANY
3. Ecrire `= ANY` est équivalent à écrire `IN`



# Sous-requêtes indépendantes avec ANY

Exemple : les produits dont le prix unitaire n'est pas minimum

```
SELECT *
FROM Produit
WHERE prixUnitaire > (
   SELECT MIN(prixUnitaire)
   FROM Produit
)
```

Autre manière d'écrire:

```
SELECT *
FROM Produit
WHERE prixUnitaire > ANY (
   SELECT prixUnitaire
   FROM Produit
)
```

# Sous-requêtes indépendantes avec ALL

```
SELECT …
FROM …
WHERE nomChamp operateur ALL (
   SELECT nomChamp2
   FROM …
   …
)
…
```

Fonctionnement :

1. Le type retourné par la sous-requête doit correspondre à celui de la requête principale
2. Le tuple de la requête principale sera gardé si le champ satisfait l'opérateur pour toutes les valeurs renvoyées par la sous requête



# Sous-requêtes indépendantes avec ALL

Exemples :

```
SELECT *
FROM Produit
WHERE prixUnitaire <= ALL (
   SELECT prixUnitaire
   FROM Produit
)
```

↳ Renvoie tous les produits dont le prix unitaire est minimum

```
SELECT *
FROM Fournisseur
WHERE id != ALL (
   SELECT idFournisseur
   FROM Produit
)
```

↳ Renvoie les fournisseurs qui ne fournissent pas de produits

Remarque : **`!= ALL`** est équivalent à **`NOT IN`**



# Sous-requêtes indépendantes avec ALL

**Avec ALL les valeurs NULL posent problème !**

1. En SQL un opérateur de comparaison peut renvoyer 3 résultats :
   - 1 > 4 renvoie FALSE
   - 1 < 7 renvoie TRUE
   - 1 < NULL renvoie UNKNOW
2. Or avec ALL, le SGBD vérifie que la comparaison renvoie TRUE **pour toutes** les valeurs renvoyées par la sous-requête.
3. Si le résultat de la sous-requête contient une valeur NULL, aucun tuple ne sera renvoyée par la requête principale.
4. Les requêtes précédentes ne fonctionnent pas s'il existe un produit dont le prixUnitaire est NULL ou dont le Fournisseur est NULL !



# Sous-requêtes indépendantes avec ALL

**Avec ALL les valeurs NULL posent problème !**

Corrections :

```
SELECT *
FROM Produit
WHERE prixUnitaire <= ALL (
   SELECT prixUnitaire
   FROM Produit
   WHERE prixUnitaire IS NOT NULL
)
```

```
SELECT *
FROM Fournisseur
WHERE id != ALL (
   SELECT idFournisseur
   FROM Produit
   WHERE idFournisseur IS NOT NULL
)
```

# Sous-requêtes corrélées

Exemple : La liste (idFournisseur, idProduit, prixUnitaire) avec le(s) produit(s) le(s) moins cher(s) de chaque fournisseur

```
SELECT idFournisseur, id AS idProduit, prixUnitaire
FROM Produit P
WHERE prixUnitaire = (
   SELECT MIN(P2.prixUnitaire)
   FROM Produit P2
   WHERE P.idFournisseur = P2.idFournisseur
)
```

Même exemple mais cette fois-ci la requête utilise **ALL** :

```
SELECT idFournisseur, id as idProduit, prixUnitaire
FROM Produit P JOIN
WHERE P.prixUnitaire <= ALL (
   SELECT prixUnitaire
   FROM Produit P2
   WHERE P.idFournisseur = P2.idFournisseur AND
         P2.prixUnitaire IS NOT NULL
)
```



# Sous-requêtes corrélées avec EXISTS

Les requêtes corrélées fonctionnent souvent avec l'opérateur EXISTS qui permet de vérifier si la sous-requête renvoie un résultat

```
SELECT …
FROM …
WHERE [ NOT ] EXISTS (
    Sous-requête
)
```

Pour la requête précédente cela donne :

```
SELECT idFournisseur, id AS idProduit, prixUnitaire
FROM Produit P
WHERE NOT EXISTS (
   SELECT *
   FROM Produit P2
   WHERE P2.prixUnitaire < P.prixUnitaire AND
         P2.idFournisseur = P.idFournisseur
)
```



# Sous-requêtes corrélées

La plupart du temps (toujours pour nous), une requête écrite avec ALL/ANY peut s'écrire avec EXISTS et aussi sans sous-requête mais avec des jointures !

En pratique s'il « sait » le faire, un SGBD remplacera les sous-requêtes par des jointures afin d'augmenter les performances.

Côté développeur, chacun a ses préférences en fonction de la question posée et de sa manière de pensée, mais nous vous demanderons souvent de proposer les 3 solutions !

Pour la requête précédente cela donne :

```
SELECT P.idFournisseur, P.id AS idProduit, P.prixUnitaire
FROM Produit P JOIN
     Produit P2 ON P.idFournisseur = P2.idFournisseur
GROUP BY P.idFournisseur, P.id , P.prixUnitaire
HAVING P.prixUnitaire = MIN(P2.prixUnitaire)
```



# Sous-requêtes exercices

Ecrire en langage SQL les requêtes permettant d'obtenir les résultats suivants (voir schéma relationnel diapo 8). Vous devrez proposer une solution avec ALL/ANY **et** une solution avec EXISTS :

1. La liste des produits dont le prix unitaire est supérieur à celui du produit d'identifiant 12
2. La liste des clients ayant commandé au moins un article le 01/09/2026
3. La liste des clients (nom, prénom) ayant passé au moins une commande le 01/09/2026 mais pas le 02/09/2026
4. La liste qui indique pour chaque client, son identifiant et la date à laquelle il a commandé le plus de produits.